

ClaimsIntel VPC - Technical Specification & Solution Architecture

This document outlines the architecture, data flows, and technical specifications for the ClaimsIntel VPC application. The solution is a deterministic, template-driven Document Intelligence platform designed to securely extract, validate, and visualize structured data from Independent Adjuster (IA) Reports, ACORD forms, and Police Reports without relying on unpredictable LLMs.

1. Technology Stack

Frontend

- **Framework:** React 18 + Vite
- **UI/UX:** Vanilla CSS (Glassmorphism design system), Lucide React (Iconography)
- **PDF Rendering:** @react-pdf-viewer/core with custom singleton plugins.
- **Key Features:** Bidirectional Spatial Bounding Box highlighting, Inline Editable Fields (Human-in-the-loop).

Backend

- **Framework:** FastAPI (Python 3.12)
 - **Document Parsing:** docling (Generates canonical document representations with pixel-perfect spatial coordinate mappings).
 - **Extraction Engine:** Declarative JSON Template Orchestrator (ported from enterprise DocIntel).
 - **Database:** SQLite (feedback.db) for secure, local VPC storage of custom schema definitions and human-in-the-loop corrections.
-

2. Solution Architecture Diagram

```
graph TD
    %% Define Styles
    classDef frontend fill:#1e293b,stroke:#3b82f6,stroke-width:2px,color:#fff
    classDef backend fill:#0f172a,stroke:#10b981,stroke-width:2px,color:#fff
    classDef logic fill:#334155,stroke:#fef08a,stroke-width:2px,color:#fff
    classDef datastore fill:#475569,stroke:#fbbf24,stroke-width:2px,color:#fff
    classDef external fill:#000000,stroke:#ef4444,stroke-width:2px,color:#fff

    %% Subgraphs for Logical Separation
    subgraph Frontend ["Frontend (React + Vite)"]
        UI["Glassmorphic UI Engine"]
        Results["Extraction Results Component"]
        ReviewFlags["Low-Confidence Alert UI"]
        Viewer["PDF Viewer Component"]
        BboxPlugin["Custom BboxPlugin<br>(Absolute CSS Overlay)"]
    end
```

```

    Editable["Inline Editable UI (Human-in-the-Loop)"]
end

subgraph Backend ["Backend (FastAPI)"]
    API["FastAPI Server (main.py)"]
    Docling["Docling Parser<br>(Generates Canonical Doc)"]

    subgraph Engine ["Declarative Extraction Engine"]
        Templates["JSON Templates<br>(e.g., police_report.json)"]
        Orchestrator["Orchestrator Core"]
        Extractors["Spatial & Regex Extractors"]
        Validation["Scoring & Validation Pipeline"]
    end

    DB[("feedback.db<br>(SQLite Local VPC Database)")]
end

subgraph FutureIntegrations ["Future Release"]
    ClaimCenter["Guidewire ClaimCenter APIs"]
end

%% Process Flow
User((Adjuster)) -->|1. Uploads PDF| API

API -->|2. Raw PDF binary| Docling
Docling -->|3. Canonical Doc (Text + BBoxes)| Orchestrator

Templates -->|4. Loads extraction strategies| Orchestrator
DB -->|5. Injects UI Custom Fields as Global Regex| Orchestrator

Orchestrator -->|6. Executes Strategies| Extractors
Extractors -->|7. Returns Candidates| Validation
Validation -->|8. Drops scores for failing rules| Orchestrator

Orchestrator -->|9. JSON Record, BBox Map, & Review Flags| Results

Results --> ReviewFlags
Results --> Editable
Results -->|10. Render Clickable Fields| UI

UI -->|11. Field Click Event| BboxPlugin
BboxPlugin -->|12. Draws Absolute Pixel Overlays| Viewer

User -->|13. Corrects Low Confidence Data| Editable
Editable -->|14. HTTP POST Correction| DB

User -->|15. Copy-pastes formatted File Note| ClaimCenter

%% Apply Styles
class UI,Results,ReviewFlags,Viewer,BboxPlugin,Editable frontend
class API,Docling backend
class Orchestrator,Templates,Extractors,Validation logic
class DB datastore

```

3. Core Engine Features

3.1. Spatial Bounding Box (BBox) Highlighting

Unlike traditional browser text-search (`ctrl+f`), which fails when the same text appears multiple times, ClaimsIntel uses deterministic pixel-mapping. The backend `docling` parser captures the absolute `[x0, y0, x1, y1]` coordinates for every parsed word. The frontend `BboxPlugin.jsx` intercepts click events, calculates percentages based on the PDF page size, and renders absolute CSS `<div>` tags as perfect highlighting overlays.

3.2. Declarative Template Engine (Orchestrator)

Extraction logic is no longer hardcoded in Python. System administrators can define extraction targets using JSON files (e.g., `backend/templates/police_report.json`). Supported Extraction Strategies:

- **global_regex:** Advanced pattern matching.
- **spatial_label:** Geometrically finds a label (e.g., "CASE NUMBER") on the PDF canvas and explicitly extracts the text located physically to its right or below it, making it highly resilient to OCR layout errors.

3.3. Scoring & Validation Pipeline

The extraction engine evaluates multiple candidates for every field. JSON templates define strict `ValidatorRules` (e.g., ensuring a Report Number is exactly 5-10 digits). If the best candidate fails validation, its confidence score drops below the `auto_accept_threshold`. This triggers a `needsReview` flag in the audit payload, prompting the frontend to render an orange `<AlertTriangle>` next to the field, forcing human verification.

3.4. Human-in-the-Loop (Dynamic Field Injection)

Adjusters can dynamically add new extraction targets directly from the UI without engineering intervention. When a new field is added, the backend instantly generates a `global_regex` strategy rule and injects it into the Orchestrator pipeline at runtime. Furthermore, any corrections made by the Adjuster via the `EditableField` components are silently logged to `feedback.db` for future AI fine-tuning.

4. Enterprise Productionization Roadmap

To transition this application from a local Proof-of-Concept into a scalable, enterprise-grade production application hosted within an organizational VPC, the following technical stack and deployment steps are required:

4.1. Required Infrastructure Upgrades

- **Containerization:** `Docker` & `Docker Compose`. Both the React Frontend and FastAPI Backend must be containerized to ensure the `docling` OCR engine and PyTorch

dependencies run consistently across all OS environments.

- **Orchestration:** Kubernetes (AWS EKS or Azure AKS) or managed container services (AWS ECS/Fargate) to handle auto-scaling during high-volume claims events (e.g., Catastrophe / CAT seasons).
- **Database Migration:** Migrate from local `sqlite3` to a NoSQL Document Database like MongoDB. MongoDB natively supports the storage of massively complex, nested JSON configurations (like our Template Schemas) and can efficiently store the Orchestrator's full extraction Audit Trails.
- **Ephemeral Storage:** Migrate the local PDF storage (`temp_filename.pdf`) to an encrypted cloud object store (Amazon S3 or Azure Blob Storage) with strict IAM policies and auto-deletion lifecycle rules (e.g., delete all PDFs 1 hour after extraction).
- **Security & Auth:** Implement a Web Application Firewall (WAF) and integrate the React frontend with an Identity Provider (Okta, Azure AD) via OAuth2/SAML so only authenticated adjusters can access the extraction UI.

4.2. CI/CD Deployment Steps

1. **Code Repository setup:** Push the codebase to an enterprise Git repository (GitHub Enterprise, GitLab, or Bitbucket) with strict branch protection rules.
2. **CI/CD Pipeline Construction:** Build automated pipelines (GitHub Actions/GitLab CI) that:
 - Run Python `pytest` and React `jest` test suites.
 - Build Docker images for both backend and frontend.
 - Scan Docker images for vulnerabilities using tools like Trivy or SonarQube.
 - Push images to a private, secure Container Registry (e.g., AWS ECR).
3. **Infrastructure as Code (IaC):** Use Terraform or AWS CloudFormation to spin up the VPC, subnets, MongoDB clusters, and Kubernetes clusters to ensure environments (Dev/Staging/Prod) are reproducible.
4. **Guidewire API Whitelisting:** Establish secure, bi-directional API gateways or VPC Peering between the ClaimsIntel VPC cluster and the Guidewire ClaimCenter instance.